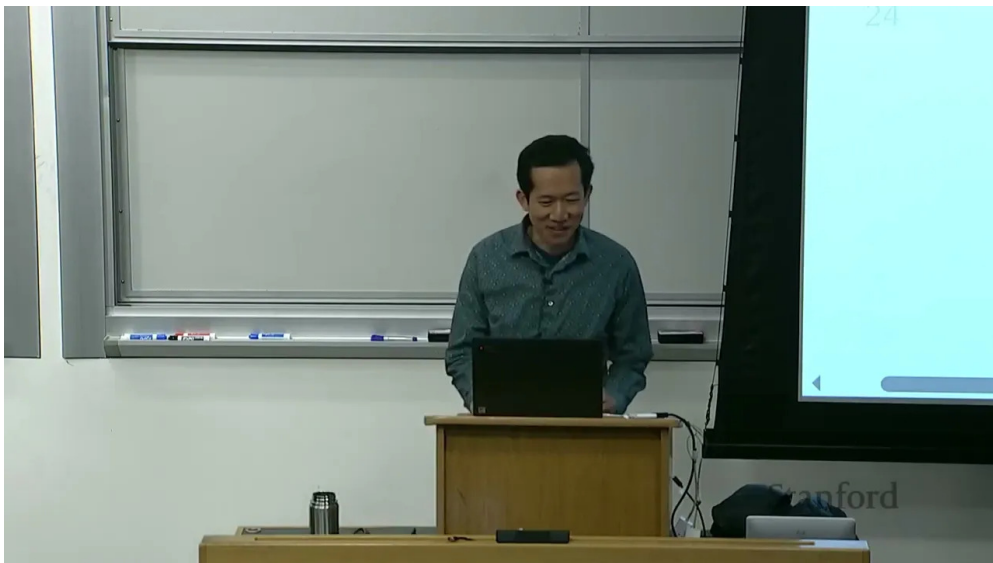


课程笔记

CS336 Lecture 1: 课程导论与整体地图

基于 lecture01.en.srt 与 lecture01-slides.py 整理

2025 年春季



课程主页: <https://stanford-cs336.github.io/spring2025/>

课程阶段: 语言模型 from scratch

本讲主题: 为什么开这门课、课程要学什么、为什么要 “build to understand”

说明: 本目录当前未提供本地 ‘slides-images’, 因此讲义以字幕稿和讲义脚本为主, 图示内容用文字整理。

目录

1 开场：这门课在讲什么	3
2 为什么要开这门课	3
3 工业化时代的语言模型	3
4 这门课能教你什么	4
4.1 经验和实验，有时比解释更诚实	4
5 Bitter Lesson 的正确理解	5
6 课程的历史背景	5
7 这门课的形式：Executable Lecture	5
8 课程整体结构	6
9 课程安排与作业	6
10 为什么特别强调效率	7
11 后续预告：下一讲会讲什么	7
12 本讲小结	7
13 补充总图：从抽象层回到完整栈	7
14 Basics 扩展：最小语言模型管线	9
14.1 Tokenization	9
14.2 Architecture	9
14.3 Training	10
15 Systems 扩展：GPU、并行和推理	11
15.1 Kernels	11
15.2 Parallelism	11
15.3 Inference	12
16 Scaling laws 扩展：把预算问题说清楚	12
16.1 compute-optimal	12
16.2 attention vs MLP	13
17 Data 扩展：把互联网变成训练语料	13
17.1 能力决定数据	13
17.2 evaluation / curation / processing	14

18 Alignment 扩展：从 raw capability 到 useful model	14
18.1 Base model vs aligned model	14
18.2 SFT	15
18.3 Preference data 与 verifiers	15
19 Tokenization 附录：把课本上的概念落到实现	16
19.1 BPE 的训练和使用	16
20 课程收尾：把这些知识拼成一个系统	17
21 基础概念总览：语言模型到底在优化什么	17
21.1 从 loss 到 perplexity	18
22 资源账本：参数、激活、优化器状态和带宽	19
22.1 一个简单的预算问题	19
23 开放性谱系：闭源、开放权重、开源	20
24 把作业当成课程的延伸	20
25 本讲再总结一次：这门课到底要你学会什么	21

1 开场：这门课在讲什么

CS336 的第一讲不是直接进入模型细节，而是先回答一个更根本的问题：为什么要在 2025 年再开一门“从零构建语言模型”的课程。讲师一开始就把课程定位得很清楚：这门课不是教你追逐最新的热点技巧，也不是让你依赖一个黑箱 API 直接调用，而是要把语言模型整个栈拆开，重新理解其数据、系统、架构和训练逻辑。

本讲的核心信息

- 课程目标不是“会用模型”，而是“理解模型如何被构建出来”。
- 课程强调完整栈：数据、tokenization、模型结构、训练、系统、规模律、对齐。
- 课程的姿态不是反对抽象，而是警惕抽象泄漏，并在必要时回到机制层面。

讲师给出的课程口号可以概括成一句话：

理解它，必须先构建它。

2 为什么要开这门课

这门课的第一层动机是“研究者正在和底层技术逐渐脱节”。更早的时候，研究者往往自己实现模型、训练模型，至少会下载一个模型并做 fine-tuning；而现在很多工作流已经变成了对一个闭源大模型直接做 prompt。这种抽象当然提高了生产力，也确实释放了大量研究效率，但它也带来了一个问题：抽象是有泄漏的。你未必真的理解系统在底层怎么工作，哪些部分是稳健的，哪些部分只是表面上能跑。

讲师把这个问题说得很直接：如果研究要继续推进，尤其是要做基础研究，就不能只停留在“字符串输入，字符串输出”的黑箱层面。很多真正有价值的问题，需要你把数据、系统、模型这些环节重新拆开，再做协同设计。

这门课的现实背景

- 研究范式已经从“自己训练模型”转向“调用大模型”。
- 这提高了效率，但也让很多人远离了真正的机制理解。
- 课程存在的理由，是让这种底层理解重新变得可学、可练、可验证。

3 工业化时代的语言模型

本讲随后把视角从学术动机转向产业现实：语言模型已经高度工业化了。课程用 GPT-4、xAI 集群、Stargate 之类的例子来说明，前沿模型的训练成本、参数规模和算力投入已经大到普通研究者无法复现。而且最关键的是，这些模型的构建细节并不公开。即使是公开的技术报告，很多时候也只会给出很有限的信息。

这意味着两件事：

- 前沿模型本身已经超出了课堂的直接可达范围。

- 但是这不妨碍我们构建“小模型”来学习机制，只是必须意识到它们未必在规模上完全代表前沿系统。

讲师用两个例子说明“规模改变一切”：

1. 在小模型里，attention 和 MLP 的 FLOPs 可能差不多；但放大到大模型后，MLP 往往成为主导项。
2. 某些能力会在规模上“涌现”，在小规模时几乎看不出来。

这两点共同指向一个结论：只在小规模上优化，不一定能优化到真正重要的地方。

More is different

规模不是简单的线性放大。很多结构、瓶颈和能力只会在更大尺度上显现出来。所以本课程既要教机制，也要教“规模意识”。

4 这门课能教你什么

讲师把课程知识分成三类：mechanics、mindset 和 intuitions。

类别	含义	可迁移性
Mechanics	机制层知识：Transformer 是什么、并行化怎么做、系统如何配合硬件	高，可系统训练
Mindset	规模意识：如何尽可能压榨硬件、如何认真对待 scaling law	高，可明确传递
Intuitions	哪些数据和建模决策真的有效	部分可传递，强依赖尺度与任务

其中 mechanics 和 mindset 是这门课最稳定、最值钱的部分。intuitions 当然也重要，但它更依赖经验、实验和尺度环境，未必能从一个尺度直接外推到另一个尺度。

4.1 经验和实验，有时比解释更诚实

在字幕里，讲师提到很多 Transformer 里的设计，未必都有漂亮的第一性原理解释。比如 SwiGLU 这类非线性，很多时候就是“实验结果确实好”，然后被采用。课程希望传达的不是“我们已经完全解释了所有设计”，而是“实验和规模经常比故事更可靠”。

一句务实的话

如果某个设计选择能在更大规模上稳定提升效果，那么它就值得重视，哪怕它一开始并没有完整的理论解释。

5 Bitter Lesson 的正确理解

讲师专门纠正了对 bitter lesson 的一种常见误读。错误理解是：既然 scale 很重要，那算法就不重要了，只要一直加大资本开模型就行。正确理解恰好相反：真正重要的是“能随着 scale 继续有效的算法”。

换句话说，最终模型效果可以粗略看成

$$\text{accuracy} \approx \text{efficiency} \times \text{resources}.$$

资源当然重要，但效率在大规模下更重要。因为当训练成本已经到数千万、数亿美元时，任何浪费都会被放大。讲师还引用了算法效率的历史改进，强调在很多任务上，算法进步本身可以带来数量级的收益，远远不只是硬件进步。

这门课的底层世界观

- 不是“算法无关”，而是“会随规模成长的算法最重要”。
- 研究的目标不是单纯堆资源，而是提高单位资源的产出。
- 问题可以被重新表述为：在给定 compute 和 data 预算下，能做出多好的模型？

6 课程的历史背景

课程随后快速回顾了语言模型的发展脉络。从 Shannon 用语言模型估计英语熵开始，到 n-gram 在机器翻译和语音识别中的应用，再到神经语言模型、seq2seq、attention、Transformer、MoE、模型并行，最后走向 GPT、BERT、T5、GPT-3、PaLM、Chinchilla，再到近年的开源/开放权重模型。

这段历史的重点不是背名词，而是看清两个趋势：

- 语言模型从“组件”变成了“核心系统”。
- 语言模型从“学术模型”变成了“工业基础设施”。

讲师还强调了开放性层次：

- 闭源模型：只能通过 API 使用。
- 开放权重模型：能看权重和部分训练细节。
- 开源模型：权重、数据和大部分细节都开放。

这对研究很重要，因为“可复现、可检查、可分析”本身就是学习的一部分。

7 这门课的形式：Executable Lecture

课程里有一个很有特色的说法：这是一门 executable lecture。意思不是“讲义里有代码”这么简单，而是：讲义本身就像一个可执行的程序，内容、代码和结构是统一的。

这有两个象征性的好处：

- 你可以把学习当作“运行一个系统”，而不是只读静态文档。

- 课程本身就体现了“构建即理解”的方法论。

1 理解系统 -> 实现系统 -> 运行系统 -> 观察系统 -> 继续改进

Listing 1: 课程式讲义的概念性示意

8 课程整体结构

本讲最后给出课程的完整地图。课程以“效率”为主线，把内容分成五个大块：

模块	关心的问题	你会学到什么
Basics	从零搭出一个基本 pipeline	tokenization、Transformer、训练循环
Systems	如何把硬件压榨到极致	kernels、并行训练、推理优化
Scaling laws	如何用小实验预测大规模表现	compute-optimal、超参外推、预算分配
Data	什么数据值得训练、如何处理数据	采集、清洗、过滤、去重、评估
Alignment	如何让 base model 真正可用	SFT、DPO、GRPO、风格和安全

这门课的核心思想是：你永远不是在抽象真空中做研究，而是在资源约束下做选择。因此，课程的每一部分都会反复回到“效率”这个关键词。

9 课程安排与作业

讲师在本讲中也给出了课程 logistics 的信息。课程作业围绕五个方向展开，对应课程的五大模块：

- 基础：实现 tokenizer、Transformer、损失函数、优化器和训练循环。
- 系统：实现 kernel、分布式训练、状态切分和性能分析。
- 规模律：在小规模实验上拟合预测大规模表现。
- 数据：从 Common Crawl 处理出适合训练的数据。
- 对齐：实现监督微调和偏好优化。

课程强调几件事：

- 没有大量脚手架代码，但会提供测试和接口。
- 先在本地验证正确性，再上集群做 benchmark。
- 一些作业有 leaderboard，目标是固定预算下做得更好。
- AI 工具可以用，但会削弱学习效果，需自己权衡。

10 为什么特别强调效率

课程把效率定义成一个总括性的概念：数据、算力、内存、带宽和算法共同决定模型能做到什么。这也解释了为什么课程安排不是按“模型论文”来排，而是按“效率瓶颈”来排。

效率驱动设计

- 数据处理要避免把计算浪费在低质量数据上。
- Tokenization 要在表达能力和计算效率之间折中。
- 模型结构要考虑内存、FLOPs 和硬件友好性。
- 训练要和预算匹配，不是越久越好。
- 对齐和数据选择也会回到资源效率问题。

11 后续预告：下一讲会讲什么

本讲最后把课程推进到下一步：tokenization。课程接下来会从最基础的部分开始，逐步把一个语言模型系统搭起来。下一讲的重点会是 PyTorch 的基本积木、资源账本，以及最小可运行 pipeline 的关键组件。

你应该带走的三句话

1. 这门课不是教你“调用模型”，而是教你“构建模型”。
2. 真正有价值的是能随着规模继续成立的算法和系统。
3. 理解语言模型，必须把数据、系统和模型一起看。

12 本讲小结

第一讲完成了课程定位，也完成了方法论铺垫。它回答了三个问题：

- 为什么要学这门课：因为前沿模型工业化后，基础理解更重要。
- 学什么：机制、规模意识和部分可迁移的直觉。
- 怎么学：从零构建一个完整系统，并持续关注效率。

如果把整门课概括成一句话，那就是：

不要只做会调用模型的人，要做知道模型为什么能工作的那个人。

13 补充总图：从抽象层回到完整栈

前面已经把课程为什么存在、为什么强调效率、为什么要把规模看成一等公民讲清楚了。这里再把这门课的“完整栈”重新压成一张图，方便后续章节反复引用。

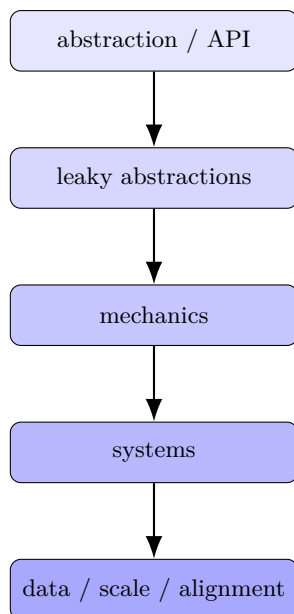


图 1: 课程反复要你做的事情: 从表层接口回到机制, 再回到可改进的完整栈

完整栈视角

这门课不是分别讲“模型”“系统”“数据”“对齐”, 而是把它们看成一个耦合系统: 哪怕你只改一个环节, 其他环节的最优解也会变。

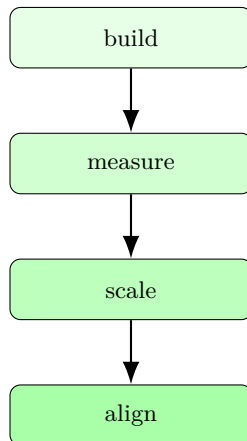


图 2: 课程的四步闭环: 先构建, 再测量, 再放大, 最后对齐到人的目标

本讲结束时应该形成的直觉

- 模型不是孤立对象, 而是资源受限系统的一部分。
- 设计不是一次性的, 而是 build-measure-scale-align 的循环。
- 研究不仅要问“能不能做”, 还要问“值不值得做”“能不能继续做大”。

14 Basics 扩展：最小语言模型管线

14.1 Tokenization

tokenization 的目标，是把原始字符串转成模型可以处理的整数序列，同时尽量控制序列长度和词表大小。字符级太大，字节级太长，词级又太稀疏，因此 BPE 成为课程中的默认选择。

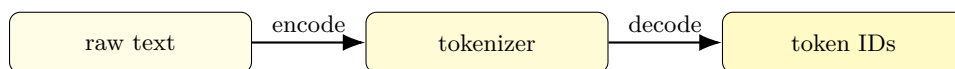


图 3: tokenizer 的双向接口：字符串与整数序列之间的 encode/decode

方案	优点	缺点
Character-based	可逆、直观	vocab 太大，稀有字符浪费
Byte-based	vocab 很小	序列太长，attention 成本高
Word-based	语义清楚	OOV 和词表爆炸
BPE	兼顾压缩率和覆盖率	需要训练和高效实现

表 1: 几种 tokenization 方案的折中

为什么 tokenization 还是必要的

因为 attention 的代价和序列长度强相关。直接用 bytes 看似优雅，但会把上下文很快吃满；直接用 words 又会让词表和稀疏性失控。

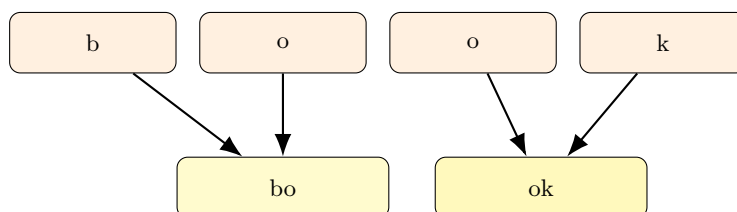


图 4: BPE 的 merge 直觉：高频相邻片段会逐步合并成更大的 token

tokenization 的工程边界

tokenization 不是“越精致越好”，而是越能压缩常见模式、同时不把序列长度拉爆越好。

14.2 Architecture

课程从原始 Transformer 出发，再讨论常见变体。每种变体都不是“为了炫技”，而是在解决表达、稳定性或效率问题。

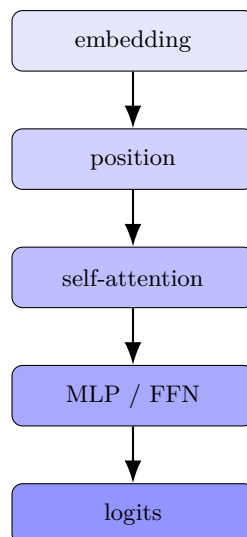


图 5: Transformer 的最小数据流：embedding、位置、attention、MLP、输出

变体	想解决什么	工程影响
SwiGLU	更强非线性	MLP 更强，常见于大模型
RoPE	位置表示更自然	长上下文外推更稳
RMSNorm	训练更稳定	比 LayerNorm 更轻
pre-norm	深层更好训练	梯度流更稳
MoE	容量与 FLOPs 解耦	用稀疏激活换大参数容量

表 2: 课程关注的架构变体和动机

架构讨论的关键标准

一个改动是否有价值，不看它有多新，而看它是否在更大规模和更高约束下继续成立。

14.3 Training

训练不是一个按钮，而是一个反馈系统：token、模型、loss 和 optimizer 构成闭环。课程会讲 optimizer、学习率 schedule、batch size、正则化和超参搜索。

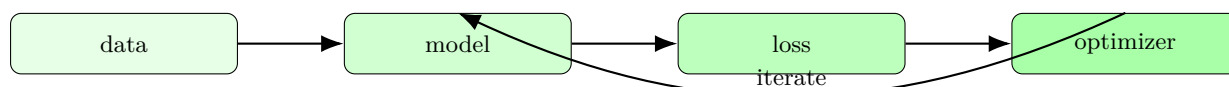


图 6: 训练闭环：训练是 feedback system，不是单一 loss 最小化

训练阶段要记住的事

- 训练不是追求 loss 越小越好，而是让有限预算下的有效学习最大化。
- 超参数耦合很强。
- 小规模有效的设置，未必能外推到大规模。

15 Systems 扩展：GPU、并行和推理

15.1 Kernels

GPU 的关键属性不是延迟，而是吞吐。kernel 设计的第一原则就是减少数据搬运，把高频数据留在更快的存储层级。

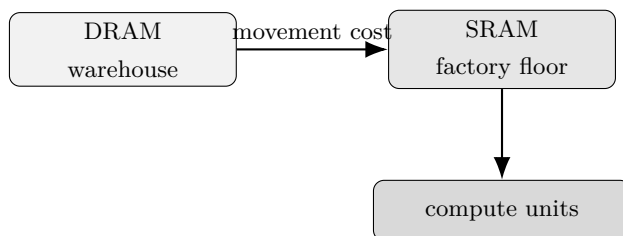


图 7: GPU/内存的类比：把计算组织到更靠近数据的位置

kernel 设计的核心不是语法

真正重要的是 memory access pattern、并行粒度和数据复用率。

15.2 Parallelism

多 GPU 并行本质上是在更高一层重新做数据流设计。你可以切 batch、切 tensor、切 layer，也可以切 sequence。核心原则没变：少搬运、重同步、合理切分。

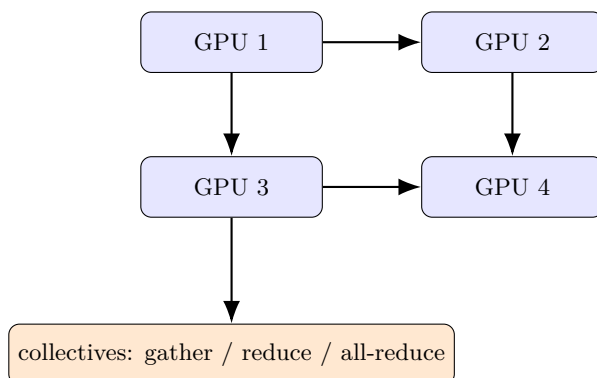


图 8: 多 GPU 并行：参数、梯度、激活和 optimizer state 都能切开

并行方式	切分对象	目的
Data parallelism	batch	提升吞吐
Tensor parallelism	matrix / attention tensors	单层太大时拆开算
Pipeline parallelism	layers	把深层网络切段
Sequence parallelism	sequence dimension	支撑更长上下文

表 3: 课程会反复出现的并行类型

15.3 Inference

推理是实际用户反复付费的地方。训练是一次性的，推理却会在产品、RL 和评测中不断发生，因此全局上推理计算常常会超过训练。

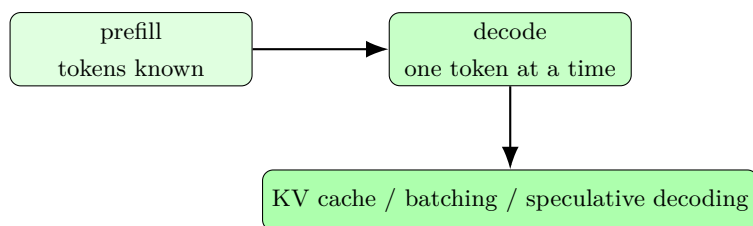


图 9: 推理的两阶段：prefill 更像训练，decode 更像逐 token 生成

推理优化的目标

让每一次生成都更便宜、更快、更稳定。

16 Scaling laws 扩展：把预算问题说清楚

16.1 compute-optimal

当 FLOPs 是硬预算时，最关键的问题就变成：是做更大的模型，还是训练更多 token？课程要教你的不是一个死公式，而是如何把它变成可测量、可拟合、可预测的问题。

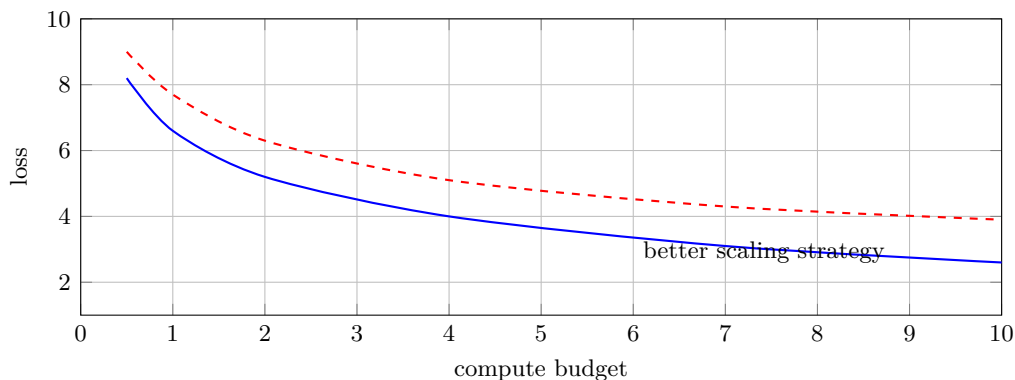


图 10: scaling law 的直觉：同样 compute 下，策略不同，loss 曲线会整体错开

问题	答案形式	意义
更大模型还是更多 token?	tradeoff	预算分配
如何做小实验?	fit a scaling law	小规模指导大规模
什么最怕浪费?	expensive compute	每一块 FLOP 都有机会成本

表 4: scaling laws 要回答的三个预算问题

scaling laws 的价值

- 让你不靠拍脑袋分配 compute。
- 让你知道实验应该做多大。
- 让你把“感觉”变成定量策略。

16.2 attention vs MLP

课程用一个朴素例子说明：在小模型里，attention 和 MLP 的 FLOPs 可能差不多；但放大以后，MLP 占比会迅速上升。把时间花在小规模 attention 的微优化上，未必会对最终大模型最关键。

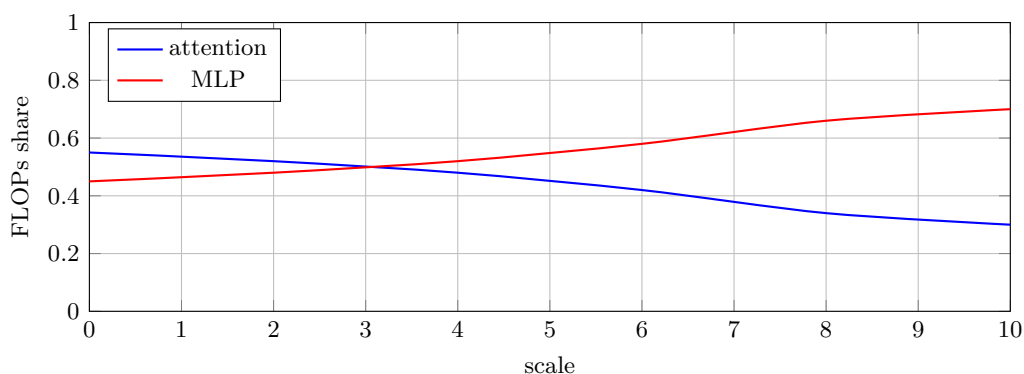


图 11: scale effect 示例：更大规模时，MLP 往往会成为更大的 FLOPs 消耗项

17 Data 扩展：把互联网变成训练语料

17.1 能力决定数据

如果你想让模型具备多语言、代码或数学能力，就必须问：数据从哪里来？怎么混配？怎么过滤？

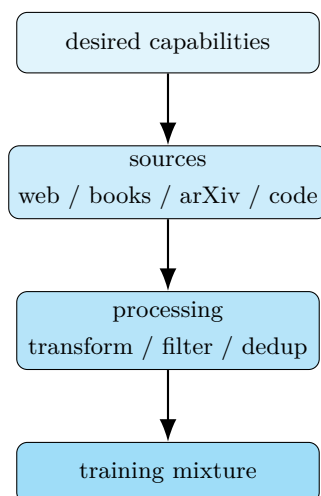


图 12: 数据工程的逻辑：目标能力决定来源，来源决定处理，处理决定混配

数据不是“抓到就行”

- 互联网上的数据是脏的、重复的、带偏差的。
- 格式不是文本而已，还包括 HTML、PDF、目录结构等。
- 你必须做转换、过滤和去重，才能让数据真的能训练。

17.2 evaluation / curation / processing

课程把 data 模块拆成三层：先知道模型会什么，再决定要什么数据，再把数据处理成可训练样本。

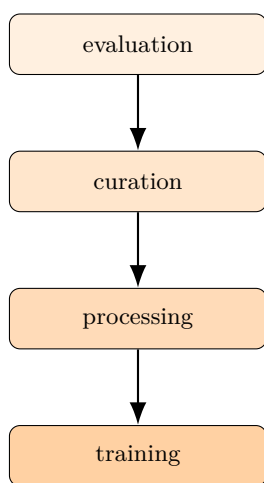


图 13: data 模块的四步链路：评估、整理、处理、训练

阶段	问题	常见方法
Evaluation	模型会不会做某类任务	perplexity、MMLU、instruction following
Curation	哪些来源值得用	web、books、code、papers、licensed data
Processing	如何变成训练样本	HTML/PDF to text、filter、dedup

表 5: data 章节要训练你的判断顺序

数据处理的第一原则

先怀疑数据，再相信模型。如果你不控制噪声、偏差和重复，模型会把这些东西原样学进去。

18 Alignment 扩展：从 raw capability 到 useful model

18.1 Base model vs aligned model

Base model 擅长完成下一 token，但并不天然适合当助手。alignment 的任务，是把 raw capability 转成真正可用、可控、可对齐人类需求的模型。

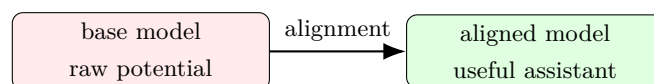


图 14: alignment 的作用：把潜在能力转成可用能力

alignment 的三个目标

- follow instructions.
- control style.
- respect safety boundaries.

18.2 SFT

SFT 的训练数据是 ‘(prompt, response)’。它的核心不是“再训一遍”，而是让模型明确知道：在这个 prompt 下，人类希望看到什么样的回答。

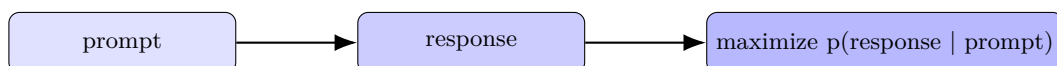


图 15: SFT 的基本结构：从 prompt-response pair 中学习条件分布

为什么 SFT 重要

- base model 可能知道答案，但不知道格式。
- SFT 把潜在能力拉到前台。
- 它是 instruction following 的第一步。

18.3 Preference data 与 verifiers

进一步的对齐不再要求每个样本都有唯一标准答案，而是让模型在多个候选中选更好的那个。这个思路可以由人类偏好，也可以由形式化 verifier 或 learned verifier 支持。

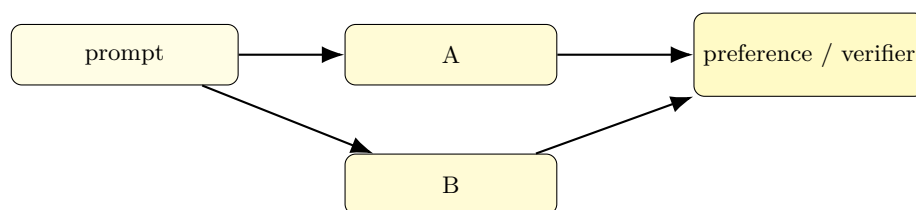


图 16: 偏好学习：比较多个候选回答，而不是只看单个标准答案

方法	对象	直觉
PPO	policy + reward model	把偏好当 RL 问题
DPO	preference pairs	直接从偏好对中学 policy
GRPO	group comparisons	用组比较减少 value function 依赖

表 6: 课程中会出现的三类偏好优化方法

verifier 的价值

对 code / math, 可以有形式化 verifier; 对更开放任务, 可以训练 learned verifier。它们的作用都是把“更好”这件事从纯人工标注中部分解放出来。

19 Tokenization 附录：把课本上的概念落到实现

19.1 BPE 的训练和使用

BPE 的训练过程是一个标准的频次合并循环：从 bytes 开始，统计相邻 pair 的出现次数，选最常见 pair 合并为新 token，重复若干轮，直到达到词表预算。编码时则把字符串逐步压成这些合并结果。

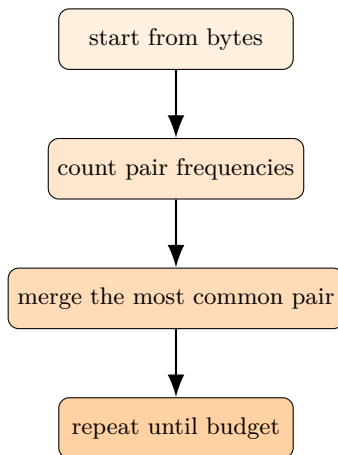


图 17: BPE 的训练循环：统计、合并、更新、重复

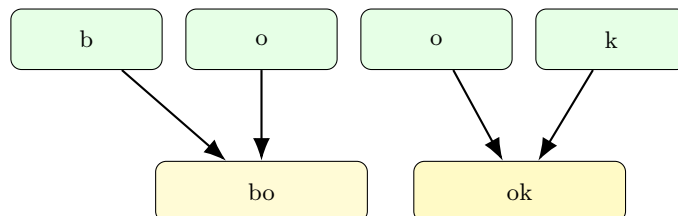


图 18: BPE 的 merge 直觉：高频相邻片段会逐步合并成更大的 token

方案	优点	缺点
Character-based	可逆、直观	vocab 太大，稀有字符浪费
Byte-based	vocab 很小	序列太长，attention 成本高
Word-based	语义清楚	OOV 和词表爆炸
BPE	兼顾压缩率和覆盖率	需要训练和高效实现

表 7: 几种 tokenization 方案的折中

tokenization 的工程边界

tokenization 不是“越精致越好”，而是越能压缩常见模式、同时不把序列长度拉爆越好。

20 课程收尾：把这些知识拼成一个系统

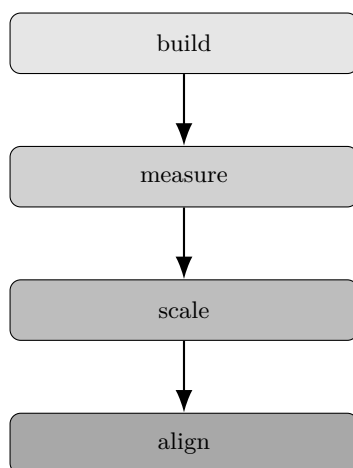


图 19: 课程给出的最终闭环：build, measure, scale, align

最后应该带走的三句话

1. 这门课不是教你“调用模型”，而是教你“构建模型”。
2. 真正有价值的是能随着规模继续成立的算法和系统。
3. 理解语言模型，必须把数据、系统和模型一起看。

最后一句话

不要只做会调用模型的人，要做知道模型为什么能工作的那个人。

21 基础概念总览：语言模型到底在优化什么

前面的内容已经回答了“为什么要学”和“课程怎么组织”。为了让后面的内容不只是概念堆叠，这里先把语言模型最核心的目标函数讲清楚。课程里虽然还没有正式进入训练细节，但如果不先知道语言模型到底在优化什么，后面的 tokenization、系统优化、规模律和对齐都很难真正连起来。

一个标准语言模型处理的是长度为 T 的 token 序列 $(x_1, \dots, x_T)$ 。它的目标不是一次性猜完整段文本，而是学习条件分布

$$p(x_t | x_{<t})$$

并在整个序列上最小化负对数似然：

$$\mathcal{L} = - \sum_{t=1}^T \log p(x_t | x_{<t}).$$

这就是 next-token prediction 的本质。

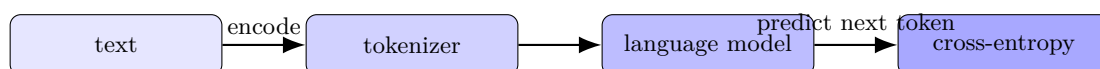


图 20: 语言模型最小闭环：原始文本经过 tokenization，再进入模型，最后用 next-token loss 训练

符号	含义	为什么重要
x_t	第 t 个 token	模型的基本预测单位
$x_{<t}$	前文上下文	决定条件概率
$p(x_t x_{<t})$	预测分布	训练和评估都围绕它展开
$\mathcal{L}$	负对数似然 / cross-entropy	真正被优化的目标

表 8: 本门课后面会反复使用的基础符号

为什么 next-token objective 很关键

它有三个好处：第一，训练信号极其密集，几乎每个 token 都能提供监督；第二，它天然适配大规模文本；第三，很多看似复杂的能力都能在这个目标下涌现出来，所以它既简单又足够强。

一个容易忽略的点

“语言模型”不是在生成“句子”，而是在估计 token 序列分布。句子只是人类解释模型输出的方式，训练目标本身更原子、更机械。

21.1 从 loss 到 perplexity

课程里经常会遇到 perplexity 这个词。它不是神秘指标，本质上就是把平均负对数似然重新指数化，便于理解“模型对下一个 token 的平均惊讶程度”。如果把交叉熵记作 H ，那么 perplexity 近似就是

$$\text{PPL} = \exp(H).$$

因此，降低 perplexity 和降低交叉熵是同一件事，只是表达方式不同。

不要把 perplexity 当成万能指标

perplexity 能反映语言建模能力，但不直接等价于“更有用”或“更安全”。后面的数据和对齐章节会不断提醒你这一点。

22 资源账本：参数、激活、优化器状态和带宽

这门课反复强调 efficiency，不是因为“省钱”这么简单，而是因为语言模型训练真正受制于一张资源账本。同样的架构放到不同硬件、不同 batch size、不同上下文长度下，主瓶颈可能完全不同。

粗略地说，一个训练系统需要同时关心四类资源：

- 参数：模型本身有多大。
- 激活：前向传播保留下来的中间状态有多少。
- 优化器状态：Adam 之类优化器会额外维护状态。
- 带宽与通信：GPU 之间、GPU 与显存之间如何移动数据。

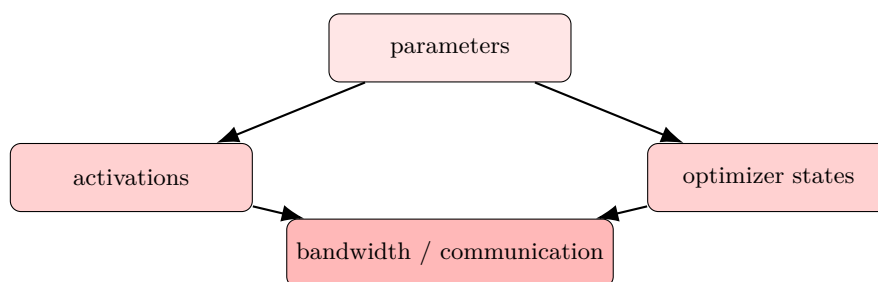


图 21: 训练中的资源账本不是单项最优化，而是参数、激活、优化器状态和带宽的共同约束

资源	常见浪费方式	课程关注的优化方向
参数	模型过大，无法放入设备	tensor/pipeline sharding
激活	上下文过长、batch 太大	sequence parallelism, checkpointing
优化器状态	Adam 状态膨胀显存占用	sharding, fused kernels
带宽	GPU 间同步过频繁	collectives, layout-aware design

表 9: 资源浪费在不同层的典型形式

为什么训练和推理的瓶颈不同

训练阶段会反复做前向和反向传播，因此更像“算力 + 显存”的联合约束；推理阶段则更容易在 decode 阶段变成“逐 token 生成 + 内存带宽”问题。

不要被“参数量”误导

参数量只是容量指标之一。真正决定系统可扩展性的，还有上下文长度、通信开销、优化器状态和推理访问模式。

22.1 一个简单的预算问题

假设你有固定预算，能做的事情通常不是“把每一项都拉满”，而是决定把预算花在模型大小、训练 token 还是系统优化上。这也是为什么这门课每一章都在回答同一个问题：在有限预算下，怎样把边际收益做大？

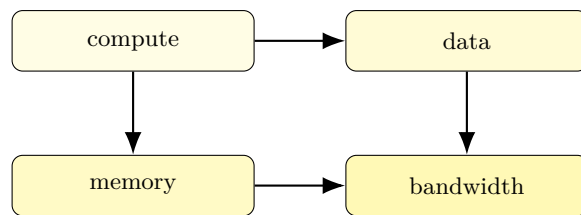


图 22: 课程中的资源约束图: compute、data、memory、bandwidth 互相牵制

23 开放性谱系：闭源、开放权重、开源

课程在历史回顾里不仅列了模型名，也强调了“开放性”本身是一条重要谱系。对研究者来说，开源/开放权重/闭源不仅是发布策略差异，更决定了你能不能检查训练细节、能不能复现实验、能不能理解失败模式。

类别	能看到什么	典型例子	研究意义
闭源	只能通过 API 使用	GPT-4 / Claude / Gemini	适合应用，但很难复现
开放权重	权重 + 部分训练信息	Llama / DeepSeek	可分析模型，仍可能缺失数据细节
开源	权重、数据、较完整细节	OLMo / 部分开源项目	最适合教学和系统研究

表 10: 开放性不是二元状态，而是一条连续谱

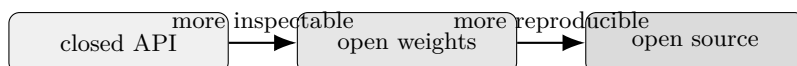


图 23: 开放性越高，越容易把“经验”变成“可验证的机制”

为什么开放性对这门课很重要

因为课程要训练的是“能独立改造系统的人”，而不是只能消费 API 的人。你越能看到训练和数据细节，就越能把直觉和工程决策对应起来。

历史回顾的真正目的

不是为了背模型名字，而是为了看清技术路线如何从“可读、可写”变成“可用但不可见”，再回到“尽可能可见、可研究”的方向。

24 把作业当成课程的延伸

这门课的作业不是脱离讲义的单独项目，而是把 lecture 中的抽象变成手上能跑的系统。如果把五个作业放在一起看，它们其实构成了一条从“最小语言模型”走到“可用模型”的路线。

作业	对应模块	你会真正做什么	最常见失败点
HW1	Basics	tokenizer, Transformer, training loop	实现正确但太慢
HW2	Systems	Triton kernel, DDP, state sharding	只对 correctness, 不对性能负责
HW3	Scaling laws	拟合和外推 scaling 曲线	小实验不稳, 外推不可信
HW4	Data	采集、转换、过滤、去重	数据质量差, 指标看不出来
HW5	Alignment	SFT, DPO, GRPO	只看 loss, 不看偏好和安全目标

表 11: 作业不是“额外负担”，而是课程主线的工程化版本

做作业时最危险的心态

只把作业看成“把分数拿到手”，而不去看它在系统层面对应什么设计问题。这样做完以后，知识不会真正积累。

做完这门课后你应该能回答的问题

- 为什么这个 tokenizer 不够好？
- 为什么这个模型在小规模上有效，在大规模上未必最优？
- 为什么系统优化能改变训练策略，而不仅是加速实现？
- 为什么数据和对齐会决定模型最终是否“有用”？

25 本讲再总结一次：这门课到底要你学会什么

如果把第一讲压缩成一张更实用的清单，答案大概是下面四项：

1. 你要能把语言模型视作一个由数据、模型、系统和目标函数组成的完整系统。
2. 你要知道为什么 tokenization、架构、训练、系统、规模律、数据和对齐不能割裂看。
3. 你要理解“效率”不是一个口号，而是决定可扩展性和可研究性的硬约束。
4. 你要开始形成一种研究姿态：先构建、再测量、再放大、最后对齐。

第一讲的真正交付物

不是某个模型细节，而是一个工作方式：从抽象入口回到机制层，再把机制放回完整工程栈里重新理解。